

Retail Transaction Analytics Platform

Overview

Retail organizations generate large volumes of transaction data containing information about products, customers, sales quantities, prices, dates, and locations. Business teams need to monitor sales performance, identify high-performing products, understand customer purchasing patterns, and track revenue trends.

However, manually combining raw transaction files, cleaning data, calculating KPIs, and preparing reports can be time-consuming and error-prone.

Retail managers often ask questions such as:

- Which products and categories generate the highest revenue?
- Which regions, months, or customer segments show the strongest sales?
- What are the trends in quantity sold, revenue, and average order value?
- Which products have declining sales and require attention?

Build a Python-based full-stack analytics application that ingests retail transaction data, performs data cleaning and transformation, stores curated data in a relational database, and provides interactive dashboards and APIs for business analysis.

Questions

- Which products and categories generate the highest revenue?
- Which regions and months have the strongest sales?
- What are the trends in revenue, quantity sold, and average order value?
- Which products show declining sales or unusual transaction patterns?

Requirement 1 – Basic

- Build a Python data-ingestion pipeline for retail transaction files
- Clean and transform transaction, product, customer, and date fields using Pandas
- Design relational tables and load curated data into PostgreSQL/MySQL
- Implement SQL queries for revenue, quantity, product, customer, and regional KPIs
- Build REST API endpoints using Flask or FastAPI for sales and product data
- Build CRUD operations for products and selected business master data
- Create an interactive dashboard using Streamlit or Flask + HTML/CSS/JavaScript
- Implement input validation, error handling, and basic logging

Requirement 2 – Advanced

- Implement incremental ETL processing for newly received transaction files
- Create reusable ETL modules for extraction, transformation, validation, and loading
- Implement data-quality checks for duplicates, missing values, invalid prices, and inconsistent dates
- Add parameterized API filtering, sorting, pagination, and date-range queries

- Implement role-based access for analyst and administrator functions
- Optimize SQL queries using indexes, aggregations, and appropriate schema design
- Add automated tests for ETL functions, APIs, and key business calculations
- Add scheduled pipeline execution and pipeline-run logging

Build a simple front-end interface.

Deliverables

1. Architecture Diagram

- Include a high-level view of the system in JPEG or PDF.
- Show how data is processed from the provided dataset through ETL, database, APIs, and the user's interface.

2. Design

- Articulate system design decisions taken and the trade-offs made.

3. Full Executable Code (Microservice)

- Focus on clarity and modularity.
- Provide a README explaining:
 - Project setup (how to install or run locally)
 - Sample usage (for example, a test API request and resulting output)
 - Database setup and schema
 - ETL execution steps

4. Panel Presentation (10 minutes)

- Demo of the working solution outlining the design and approach (8 minutes).
- Q&A (2 minutes).

Dataset Name

Online Retail II

Primary Link

UCI Machine Learning Repository – Online Retail II

<https://archive.ics.uci.edu/dataset/502/online+retail>

Alternative Link

UCI Online Retail Dataset

<https://archive.ics.uci.edu/dataset/352/online+retail>

Format

Excel / CSV

Key Fields

InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country

Reason

The dataset contains transaction-level retail records with product, quantity, price, customer, country, and date fields. It supports ingestion, cleaning, transformation, SQL modeling, KPI calculation, API development, dashboarding, and incremental ETL exercises.